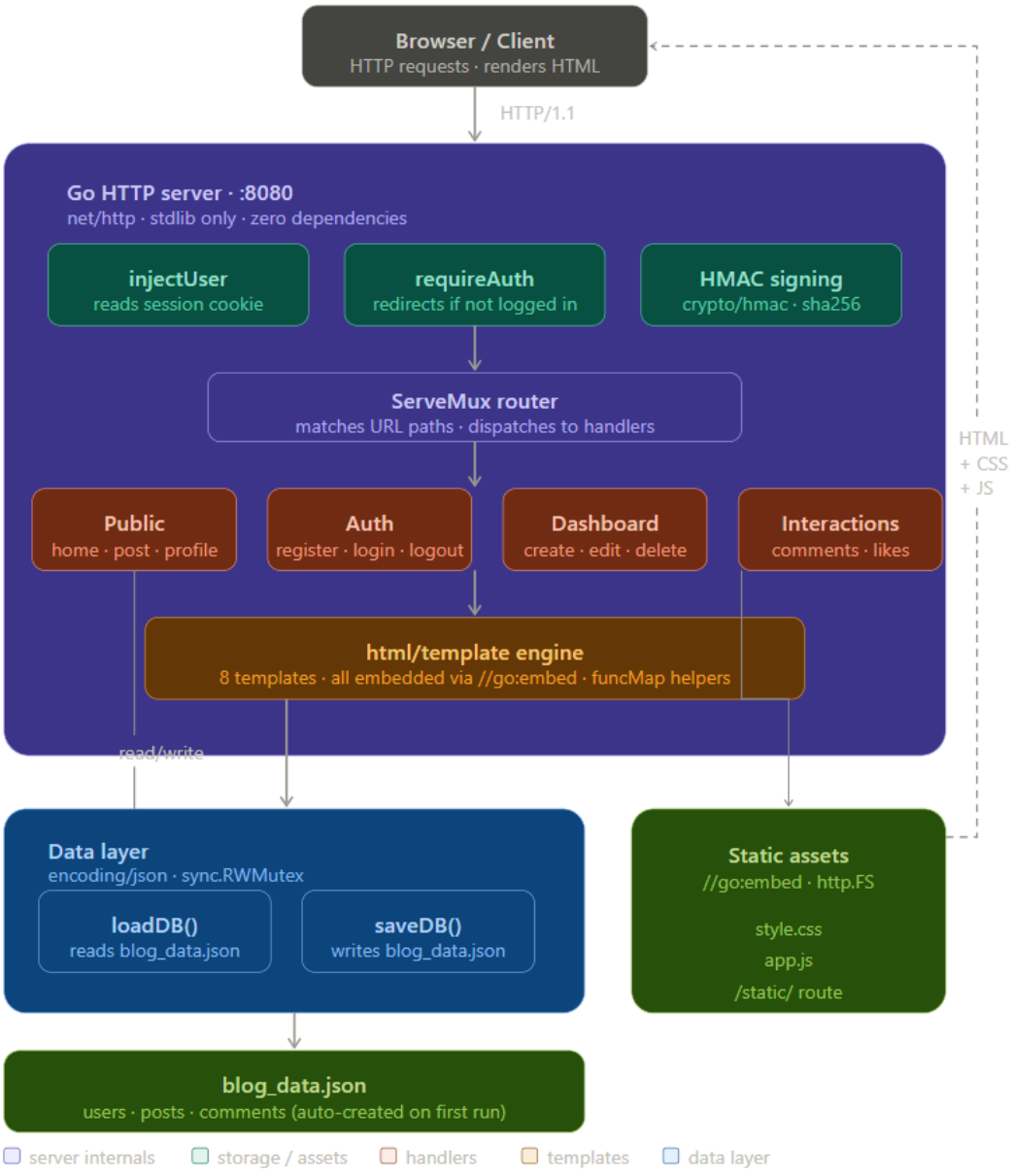
High- Level System Diagram



1. Browser/Client

It is like a customer sitting at a desk, about to order something from a restaurant. Here in the Goblog platform ,

HTTP GET → Server → HTML response → browser renders it

2. HTTP/1.1 arrow

We can take this as a telephone line connecting people who only speak one agreed language. Here HTTP is the language between browser and client.
In the code we can see,

GET/ post/abc HTTP/1.1
Host: localhost:8080 → 200 OK + HTML body

3. Go HTTP server: 8080

This is like a restaurant building. Everything that happens- middleware, router, handlers, templates and data live inside this building.

http.ListenAndServe(":8080", mux) // means like Opens the door and starts accepting visitors.

4. injectUser middleware

A host at the restaurant entrance who checks if you have a Loyalty card in your wallet. That card is the session cookie.

Cookie "s" → HMAC verify → ctx.WithValue(userID, username) // Attaches identity to every request, silently.

5. requireAuth middleware

This is like a rope barrier with a bouncer. Members-only area beyond it. This is the redirect to /login that non-logged-in users see when they try to visit /dashboard.

```
If userID == " " {
redirect → /login
  Return
   }          //only logged-in users pass
```

6. HMAC signing

A letter sealed with a wax stamp that only the restaurant owner can create. When you first login, the server hands you a loyalty card with a wax seal on the back. Only the server knows the secret stamp pattern. Next time you visit, the host flips the card over, check the seal pattern, and knows instantly whether the card is genuine or if someone tried to forge it. HMAC is that secret stamp.

```
Hmac.New(sha256.New, secretKey)
// Signs : userID?|username|timestamp
// verifies on every request- unforgeable without the key.
```

7. ServeMux router

A head waiter with a sealing chart, pointing different guests to different sections of the restaurant. When a request arrives, it has URL like post/abc or /dashboard/post/new. The router is the head waiter with a seating chart. It looks at the URL and says: Ah, /post/requests go to the Reading section, /dashboard/requests go to the Members section, /static/requests go to the shelf by the door. It just directs- it doesn't serve anything itself.

```
mux.HandleFunc("/post/", …)
mux.HandleFunc("/dashboard", …)
mux.Handle("/static/", …)
```

8. Handlers (Public.Auth.Dashboard.Interactions)

This means four different waiters, each expert in their section: the front desk, the members lounge, the public dining room, and the bar. The public waiter serves anyone – Home page, individual posts, author profiles. The Auth waiter handles signing up and logging up. The Dashboard waiter only serves members creating and editing posts. The Interactions waiter handles likes and comments.

homeHandler / viewPostHandler

registerHandler / loginHandler

newPostHandler / editPostHandler

commentHandler / likeHandler

9. Html/ template + blog_data.json

It is like a printer that takes a template menu and fills in today's specials from the chef's notebook. The template engine is a menu printer. It holds blank menu templates(home.html, post.html, dashboard.html).When a handler is ready to respond, it grabs the right template and fills in the blanks with real data – post titles , author names, comments- read from blog_data.json, which is the chef's notebook of everything the restaurant knows. The filled-in menu (HTML page) goes back to the customer.

loadDB() → posts, users,comments

template.ParseFiles("post.html")

t.Execute(w, data) //Merges

template + data → HTML